Q1. Employee Performance: Population vs. Sample Variance A company records the performance scores of eight employees.

```
# (a) Write Python code to calculate the mean, population variance, sample variance, population
#  standard deviation, and sample standard deviation. Implement the variance and standard deviation
#  formulas manually without using np.var() or np.std() for the calculations.
scores = [45, 50, 52, 55, 58, 60, 65, 75]

def calculate_mean(data):
    return sum(data) / len(data)

def population_variance(data):
    mean = calculate_mean(data)
    total = 0
    for x in data:
        total += (x - mean) ** 2

    return total / len(data)

def sample_variance(data):
    mean = calculate_mean(data)
    total = 0

    for x in data:
        total += (x - mean) ** 2

    return total / (len(data) - 1)

# Standard deviations
def population_std(data):
    return population_variance(data) ** 0.5

def sample_std(data):
    return sample_variance(data) ** 0.5

# Display results
print("Mean:", calculate_mean(scores))
print("Population Variance:", population_variance(scores))
print("Sample Variance:", sample_variance(scores))
print("Population Standard Deviation:", population_std(scores))
print("Sample Standard Deviation:", sample_std(scores))
```

```
Mean: 57.5
Population Variance: 77.25
Sample Variance: 88.28571428571429
Population Standard Deviation: 8.789197915623474
Sample Standard Deviation: 9.396047801374484
```

```
# (b) Explain why the population and sample variance differ. Quantify the difference using the calculated results and recom
# if these eight employees represent a sample of a much larger workforce.
1. Why do they differ?

Population variance divides the sum of squared deviations by \(N\), while sample variance divides it by \(n-1\).
- Population variance: Used when the data contains every member of the population.
- Sample variance: Uses \(n-1\) (Bessel's correction) to provide an unbiased estimate of population variance when the popul
2. Numerical comparison

For the given scores [45, 50, 52, 55, 58, 60, 65, 75]:
- Number of employees: \(n=8\)
- Mean: \(57.5\)
- Sum of squared deviations: \(618\)
Measure Calculation Result
Population variance \(618/8\)   77.25
Sample variance \(618/7\)   88.29
Difference  \(88.29-77.25\) 11.04

Percentage difference relative to population variance:
\[
\frac{88.2857-77.25}{77.25}\times100
\approx 14.28\%
\]
Thus, sample variance is approximately 11.04 points², or 14.28%, higher than population variance.

3. Recommendation

If these eight employees represent a sample of a much larger workforce, use the sample variance formula, dividing by \(n-1
```

Q2. Customer Spending: Comparing Data Spread Use Seaborn's built-in tips dataset. Compare the total_bill and tip columns.

```
# (a) Calculate the population variance and population standard deviation for both columns
# . Then calculate their means and coefficients of variation (CV). Use Python to compare their absolute and relative variat
import seaborn as sns
import numpy as np

df = sns.load_dataset("tips")
total_bill = df["total_bill"]
tip = df["tip"]

for name, data in [("Total Bill", total_bill), ("Tip", tip)]:
    mean = np.mean(data)
    variance = np.var(data, ddof=0)
    std_dev = np.std(data, ddof=0)
    cv = (std_dev / mean) * 100

    print(name)
    print("Mean:", round(mean, 2))
    print("Population Variance:", round(variance, 2))
    print("Population Standard Deviation:", round(std_dev, 2))
    print("Coefficient of Variation:", round(cv, 2), "%")
    print()
```

```
Total Bill
Mean: 19.79
Population Variance: 78.93
Population Standard Deviation: 8.88
Coefficient of Variation: 44.9 %

Tip
Mean: 3.0
Population Variance: 1.91
Population Standard Deviation: 1.38
Coefficient of Variation: 46.05 %
```

```
# (b) Identify the five largest total_bill values and calculate how the mean, variance, and standard
# deviation change when those observations are temporarily excluded. Explain why variance can be more sensitive to extreme
df = sns.load_dataset("tips")

original = df["total_bill"]

largest_five = original.nlargest(5)

print("Five largest total bills:")
print(largest_five.to_list())

reduced = original.drop(largest_five.index)
for name, data in [
    ("Original Data", original),
    ("After Excluding Five Largest Bills", reduced)
]:
    mean = np.mean(data)
    variance = np.var(data, ddof=0)
    std_dev = np.std(data, ddof=0)

    print("\n", name)
    print("Number of observations:", len(data))
    print("Mean:", round(mean, 2))
    print("Population Variance:", round(variance, 2))
    print("Population Standard Deviation:", round(std_dev, 2))

# Calculate the change in statistics
print("\nChange after exclusion:")
print("Mean change:", round(reduced.mean() - original.mean(), 2))
print("Variance change:",
      round(np.var(reduced, ddof=0) - np.var(original, ddof=0), 2))
print("Standard deviation change:",
      round(np.std(reduced, ddof=0) - np.std(original, ddof=0), 2))
```

```
Five largest total bills:
[50.81, 48.33, 48.27, 48.17, 45.35]

 Original Data
Number of observations: 244
Mean: 19.79
Population Variance: 78.93
Population Standard Deviation: 8.88

 After Excluding Five Largest Bills
Number of observations: 239
Mean: 19.19
```

```
Population Variance: 63.29
Population Standard Deviation: 7.96

Change after exclusion:
Mean change: -0.59
Variance change: -15.64
Standard deviation change: -0.93
```

Q3. Build a Variance and Standard Deviation Calculator Create a reusable Python program for the following datasets.

```
# (a) Implement separate functions to calculate the mean, population variance, sample variance
# , population standard deviation, and sample standard deviation without using NumPy's built-in statistical calculation fur
data_1 = [10, 20, 30, 40, 50]
data_2 = [20, 20, 20, 20, 20]
data_3 = [5, 10, 15, 20, 100]
data_4 = [12, 14, 16, 18, 20, 22]

def calculate_mean(data):
    if len(data) == 0:
        raise ValueError("Dataset cannot be empty")
    return sum(data) / len(data)

def population_variance(data):
    mean = calculate_mean(data)
    total = 0

    for x in data:
        total += (x - mean) ** 2

    return total / len(data)


def sample_variance(data):
    if len(data) < 2:
        raise ValueError("Sample variance requires at least 2 values")

    mean = calculate_mean(data)
    total = 0

    for x in data:
        total += (x - mean) ** 2

    return total / (len(data) - 1)

def population_std(data):
    return population_variance(data) ** 0.5

def sample_std(data):
    return sample_variance(data) ** 0.5

datasets = {
    "Dataset 1": [12, 15, 15, 18, 20, 22, 25],
    "Dataset 2": [5, 5, 10, 10, 15, 20],
    "Dataset 3": [7, 14, 21, 28, 35],
    "Dataset 4": [100, 100, 100, 100],
    "Dataset 5": [1, 2, 3, 4, 5, 100]
}

print("=" * 76)
print(f"{'Dataset':<12}{'Mean':>9}{'Pop.Var':>12}"
      f"{'Sample Var':>13}{'Pop.Std':>12}{'Sample Std':>13}")
print("=" * 76)

for name, data in datasets.items():
    print(
        f"{name:<12}"
        f"{calculate_mean(data):>9.2f}"
        f"{population_variance(data):>12.2f}"
        f"{sample_variance(data):>13.2f}"
        f"{population_std(data):>12.2f}"
        f"{sample_std(data):>13.2f}"
    )

print("=" * 76)
```

```
============================================================================
Dataset          Mean     Pop.Var   Sample Var     Pop.Std   Sample Std
============================================================================
Dataset 1       18.14       17.55        20.48        4.19         4.53
Dataset 2       10.83       28.47        34.17        5.34         5.85
Dataset 3       21.00       98.00       122.50        9.90        11.07
```

```
Dataset 4      100.00        0.00         0.00        0.00         0.00
Dataset 5       19.17     1308.47      1570.17       36.17        39.63
============================================================================
```

```
# (b) Test all four datasets and produce a formatted report. Identify the dataset with zero variability
# and the dataset most affected by an extreme observation. Justify your conclusions using the calculated variance and stanc
datasets = {
    "Dataset 1": [12, 15, 15, 18, 20, 22, 25],
    "Dataset 2": [5, 5, 10, 10, 15, 20],
    "Dataset 3": [7, 14, 21, 28, 35],
    "Dataset 4": [100, 100, 100, 100],
    "Dataset 5": [1, 2, 3, 4, 5, 100]
}

# Formatted report
print("=" * 76)
print(f"{'Dataset':<12}{'Mean':>9}{'Pop.Var':>12}"
      f"{'Sample Var':>13}{'Pop.Std':>12}{'Sample Std':>13}")
print("=" * 76)

for name, data in datasets.items():
    print(
        f"{name:<12}"
        f"{calculate_mean(data):>9.2f}"
        f"{population_variance(data):>12.2f}"
        f"{sample_variance(data):>13.2f}"
        f"{population_std(data):>12.2f}"
        f"{sample_std(data):>13.2f}"
    )
# Conclusions
# 1. Zero variability — Dataset 4: Every value is 100. Therefore, every deviation from the mean
#  is zero, making both population and sample variance and standard deviation equal to zero.

# 2. Most affected by an extreme observation — Dataset 5: The value 100 is much larger than
# the other observations (1, 2, 3, 4, 5). It increases the mean and causes a large squared deviation, producing much higher

# 3. Population vs. sample: Population variance divides the sum of squared deviations by
# \(N\), while sample variance divides by \(n-1\). Therefore, for a dataset with at least two observations, sample variance

# The key lesson is that variance and standard deviation measure spread, and an extreme observation can increase both subst
```

```
============================================================================
Dataset          Mean     Pop.Var   Sample Var     Pop.Std   Sample Std
============================================================================
Dataset 1       18.14       17.55        20.48        4.19         4.53
Dataset 2       10.83       28.47        34.17        5.34         5.85
Dataset 3       21.00       98.00       122.50        9.90        11.07
Dataset 4      100.00        0.00         0.00        0.00         0.00
Dataset 5       19.17     1308.47      1570.17       36.17        39.63
```

Q4. Investigating the Effect of Mathematical Transformations A researcher records these measurements:

```
# (a) Create two new datasets: one by adding 10 to every observation and another by multiplying every
#  observation by 4. Calculate the mean, population variance, and population standard deviation for all three datasets.
#  Use Python to verify the mathematical relationships between the original and transformed statistics.
measurements = [12, 15, 18, 20, 25, 30, 40]

add_10 = [x + 10 for x in measurements]
multiply_4 = [x * 4 for x in measurements]
multiply_minus_2 = [x * -2 for x in measurements]

def calculate_mean(data):
    return sum(data) / len(data)


def population_variance(data):
    mean = calculate_mean(data)
    total = 0

    for x in data:
        total += (x - mean) ** 2

    return total / len(data)


def population_std(data):
    return population_variance(data) ** 0.5

datasets = {
    "Original": measurements,
    "Add 10": add_10,
```

```
    "Multiply by 4": multiply_4,
    "Multiply by -2": multiply_minus_2
}

print(f"{'Dataset':<18}{'Mean':>12}{'Variance':>14}{'Std Dev':>12}")
print("-" * 56)

for name, data in datasets.items():
    print(
        f"{name:<18}"
        f"{calculate_mean(data):>12.2f}"
        f"{population_variance(data):>14.2f}"
        f"{population_std(data):>12.2f}"
    )

original_mean = calculate_mean(measurements)
original_var = population_variance(measurements)
original_std = population_std(measurements)

print("\nVerification:")
print("Mean after adding 10:",
      round(calculate_mean(add_10), 2),
      "Expected:", round(original_mean + 10, 2))

print("Variance after adding 10:",
      round(population_variance(add_10), 2),
      "Expected:", round(original_var, 2))

print("Variance after multiplying by 4:",
      round(population_variance(multiply_4), 2),
      "Expected:", round(original_var * 16, 2))

print("Std dev after multiplying by 4:",
      round(population_std(multiply_4), 2),
      "Expected:", round(original_std * 4, 2))
```

```
Dataset                  Mean      Variance     Std Dev
--------------------------------------------------------
Original                22.86         80.12        8.95
Add 10                  32.86         80.12        8.95
Multiply by 4           91.43       1281.96       35.80
Multiply by -2         -45.71        320.49       17.90

Verification:
Mean after adding 10: 32.86 Expected: 32.86
Variance after adding 10: 80.12 Expected: 80.12
Variance after multiplying by 4: 1281.96 Expected: 1281.96
Std dev after multiplying by 4: 35.8 Expected: 35.8
```

```
# b) Create a fourth dataset by multiplying every observation by \(-2\). Calculate its mean, variance,
#  and standard deviation. Explain why a negative multiplier changes the direction of the values but does not make variance

measurements = [12, 15, 18, 20, 25, 30, 40]

new_data = [x * -2 for x in measurements]

def calculate_mean(data):
    return sum(data) / len(data)

def population_variance(data):
    mean = calculate_mean(data)
    total = 0

    for x in data:
        total += (x - mean) ** 2

    return total / len(data)

def population_std(data):
    return population_variance(data) ** 0.5

print("New dataset:", new_data)
print("Mean:", round(calculate_mean(new_data), 2))
print("Population Variance:", round(population_variance(new_data), 2))
print("Population Standard Deviation:",
      round(population_std(new_data), 2))
```

```
New dataset: [-24, -30, -36, -40, -50, -60, -80]
Mean: -45.71
Population Variance: 320.49
Population Standard Deviation: 17.9
```

Q5. Outlier Detection and Robust Spread Analysis A data analyst records daily sales, in thousands of rupees.

```
# (a) Write Python code to calculate the mean, median, population variance, and population standard deviation.
#  Then identify potential outliers using the IQR method and report the quartiles, IQR, lower bound, upper bound, and flagg
sales = [    12, 14, 15, 15, 16, 17, 18,    19, 20, 21, 22, 24, 25, 120]

def calculate_mean(data):
    return sum(data) / len(data)

def calculate_median(data):
    data = sorted(data)
    n = len(data)
    mid = n // 2

    if n % 2 == 1:
        return data[mid]
    else:
        return (data[mid - 1] + data[mid]) / 2

def population_variance(data):
    mean = calculate_mean(data)
    total = 0

    for x in data:
        total += (x - mean) ** 2

    return total / len(data)

def population_std(data):
    return population_variance(data) ** 0.5

sorted_sales = sorted(sales)
n = len(sorted_sales)

lower_half = sorted_sales[:n // 2]
upper_half = sorted_sales[n // 2:]

Q1 = calculate_median(lower_half)
Q3 = calculate_median(upper_half)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = [
    x for x in sales
    if x < lower_bound or x > upper_bound
]

print("Mean:", round(calculate_mean(sales), 2))
print("Median:", calculate_median(sales))
print("Population Variance:", round(population_variance(sales), 2))
print("Population Standard Deviation:", round(population_std(sales), 2))

print("\nQ1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower Bound:", lower_bound)
print("Upper Bound:", upper_bound)
print("Potential Outliers:", outliers)
```

```
Mean: 25.57
Median: 18.5
Population Variance: 699.39
Population Standard Deviation: 26.45

Q1: 15
Q3: 22
IQR: 7
Lower Bound: 4.5
Upper Bound: 32.5
Potential Outliers: [120]
```

```
# (b) Temporarily exclude the flagged outlier and recalculate the mean, median, variance, and standard deviation.
#  Compare the percentage changes in variance and standard deviation, and recommend which statistics best describe typical
# daily sales while explaining why an extreme observation should be investigated before removal.

clean_sales = [x for x in sales if x not in outliers]

print("\nComparison Before and After Removing Outlier")
```

```
for name, data in [
    ("Original", sales),
    ("Without Outlier", clean_sales)
]:
    print("\n", name)
    print("Mean:", round(calculate_mean(data), 2))
    print("Median:", round(calculate_median(data), 2))
    print("Population Variance:",
          round(population_variance(data), 2))
    print("Population Standard Deviation:",
          round(population_std(data), 2))

# Percentage changes
old_var = population_variance(sales)
new_var = population_variance(clean_sales)

old_std = population_std(sales)
new_std = population_std(clean_sales)

variance_change = (new_var - old_var) / old_var * 100
std_change = (new_std - old_std) / old_std * 100

print("\nPercentage Change in Variance:",
      round(variance_change, 2), "%")
print("Percentage Change in Standard Deviation:",
      round(std_change, 2), "%")
```

```
Comparison Before and After Removing Outlier

 Original
Mean: 25.57
Median: 18.5
Population Variance: 699.39
Population Standard Deviation: 26.45

 Without Outlier
Mean: 18.31
Median: 18
Population Variance: 14.52
Population Standard Deviation: 3.81

Percentage Change in Variance: -97.92 %
Percentage Change in Standard Deviation: -85.59 %
```

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.